



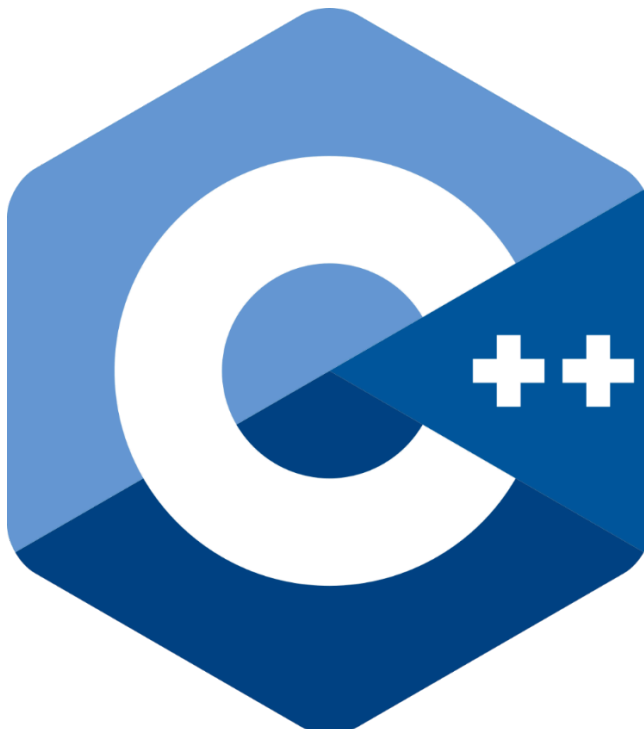
HITEC UNIVERSITY, TAXILA

**DEPARTMENT OF COMPUTER
ENGINEERING**

Lab # 01

Introduction to C++

Basics



Dated:

Week 1

Semester:

Fall 2022



What is C++

C++ is a general purpose, case-sensitive, free-form programming language that supports object-oriented, procedural and generic programming.

C++ is a middle-level language, as it encapsulates both high and low level language features.

C++ history

History of C++ language is interesting to know. Here we are going to discuss brief history of C++ language.

C++ programming language was developed in 1980 by Bjarne Stroustrup at bell laboratories of AT&T (American Telephone & Telegraph), located in U.S.A.

Bjarne Stroustrup is known as the **founder of C++ language**.



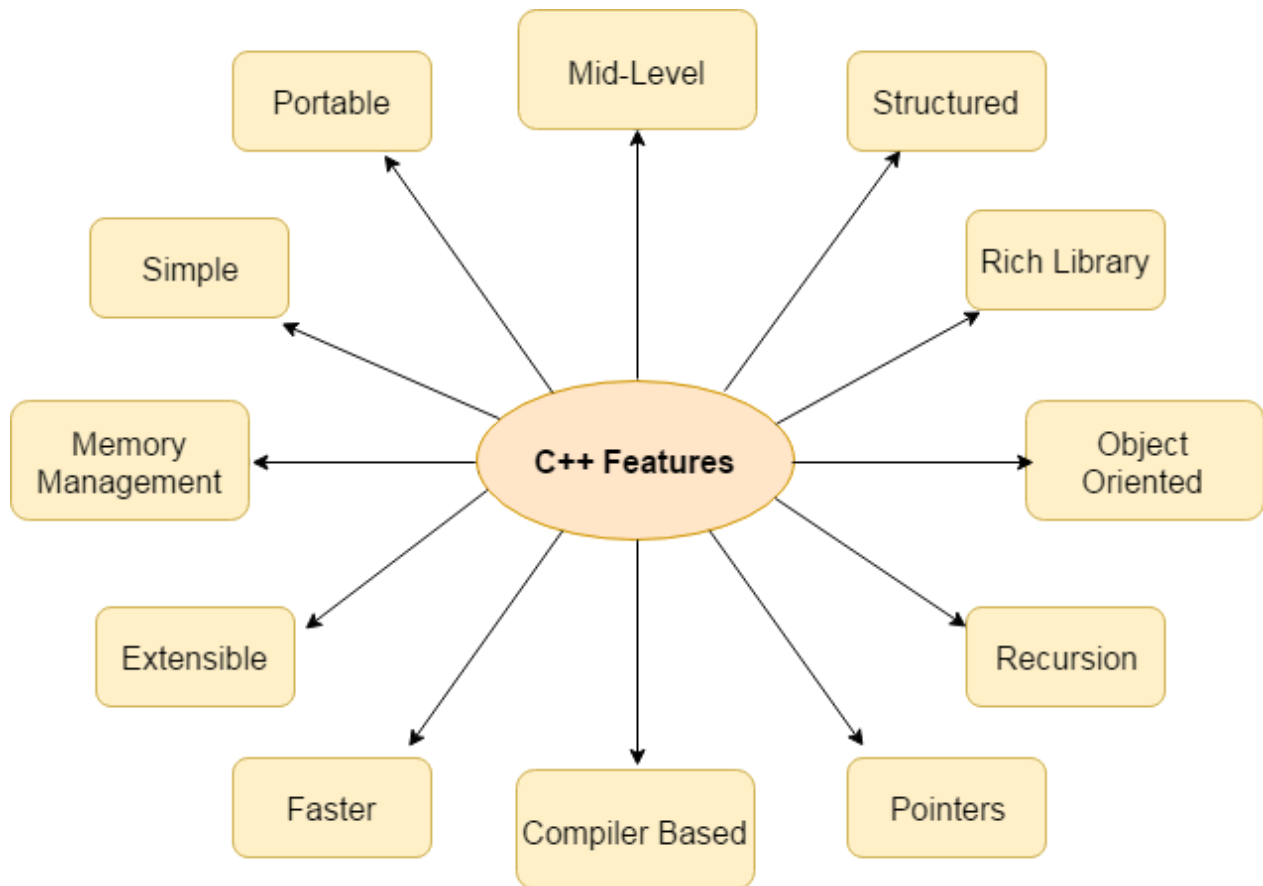
It was develop for adding a feature of **OOP (Object Oriented Programming)** in C without significantly changing the C component.

C++ programming is "relative" (called a superset) of C, it means any valid C program is also a valid C++ program.

Let's see the programming languages that were developed before C++ language.

C++ Features:

C++ is object oriented programming language. It provides a lot of **features** that are given below.



Applications of C++ Programming:

As mentioned before, C++ is one of the most widely used programming languages. It has its presence in almost every area of software development. I'm going to list a few of them here:

- **Application Software Development** - C++ programming has been used in developing almost all the major Operating Systems like Windows, Mac OSX and Linux. Apart from the operating systems, the core part of many browsers like Mozilla Firefox and Chrome have been written using C++. C++ also has been used in developing the most popular database system called MySQL.



- **Programming Languages Development** - C++ has been used extensively in developing new programming languages like C#, Java, JavaScript, Perl, UNIX's C Shell, PHP and Python, and Verilog etc.
- **Computation Programming** - C++ is the best friends of scientists because of fast speed and computational efficiencies.
- **Games Development** - C++ is extremely fast which allows programmers to do procedural programming for CPU intensive functions and provides greater control over hardware, because of which it has been widely used in development of gaming engines.
- **Embedded System** - C++ is being heavily used in developing Medical and Engineering Applications like softwares for MRI machines, high-end CAD/CAM systems etc.

First program is C++:

```
// This is my first program is C++  
/* this program will illustrate different  
components of a simple program in C++ */  
  
#include  
<iostream> using  
namespace std;  
  
int main()  
{  
    cout << "Hello  
    World!"; return 0;  
}
```

When the above program is compiled, linked and executed, the following output is displayed on the VDU screen.

Hello World!



Various components of this program are discussed below:

Comments

First three lines of the above program are comments and are ignored by the compiler. Comments are included in a program to make it more readable. If a comment is short and can be accommodated in a single line, then it is started with double slash sequence in the first line of the program. However, if there are multiple lines in a comment, it is enclosed between the two symbols `/*` and `*/`

`#include <iostream>`

The line in the above program that start with `#` symbol are called directives and are instructions to the compiler. The word include with `'#'` tells the compiler to include the file `iostream` into the file of the above program. File `iostream` is a header file needed for input/ output requirements of the program. Therefore, this file has been included at the top of the program.

`using namespace std;`

All the elements of the standard C++ library are declared within `std`. This line is very frequent in C++ programs that use the standard library.

`int main ( )`

The word `main` is a function name. The brackets `( )` with `main` tells that `main ( )` is a function. The word `int` before `main ( )` indicates that integer value is being returned by the function `main ( )`. When program is loaded in the memory, the control is handed over to function `main ( )` and it is the first function to be executed.

`Curly bracket and body of the function main ( )`

A C++ program starts with function called `main()`. The body of the function is enclosed between curly braces. The program statements are written within the brackets. Each statement must end by a semicolon, without which an error message is generated.



```
cout<<"Hello World!";
```

This statement prints our "Hello World!" message on the screen. cout understands that anything sent to it via the << operator should be printed on the screen.

```
return 0;
```

This is a new type of statement, called a return statement. When a program finishes running, it sends a value to the operating system. This particular return statement returns the value of 0 to the operating system, which means “everything went okay!”.

C++ Basic Input/ Output

C++ I/O operation is using the stream concept. Stream is the sequence of bytes or flow of data. It makes the performance fast.

If bytes flow from main memory to device like printer, display screen, or a network connection, etc, this is called as **output operation**.

If bytes flow from device like printer, display screen, or a network connection, etc to main memory, this is called as **input operation**.

I/O Library Header Files

Let us see the common header files used in C++ programming are:

Header File	Function and Description
<iostream>	It is used to define the cout , cin and cerr objects, which correspond to standard output stream, standard input stream and standard error stream, respectively.



<code><iomanip></code>	It is used to declare services useful for performing formatted I/O, such as setprecision and setw .
<code><fstream></code>	It is used to declare services for user-controlled file processing.

Standard output stream (cout)

The **cout** is a predefined object of **ostream** class. It is connected with the standard output device, which is usually a display screen. The cout is used in conjunction with stream insertion operator (<<) to display the output on a console

Let's see the simple example of standard output stream (cout):

```
1. #include <iostream>
2. using namespace std;
3. int main() {
4.     string abc = "Welcome to C++ tutorial";
5.     cout << "Output is: " << abc << endl; 6. }
```

Output:

Output is: Welcome to C++ tutorial

Standard input stream (cin)

The **cin** is a predefined object of **istream** class. It is connected with the standard input device, which is usually a keyboard. The cin is used in conjunction with stream extraction operator (>>) to read the input from a console.

Let's see the simple example of standard input stream (cin):

```
1. #include <iostream>
2. using namespace std;
3. int main() {
4.     int age;
5.     cout << "Enter your age: ";
```



6. `cin >> age;`
7. `cout << "Your age is: " << age << endl; 8. }`

Output:

```
Enter your age: 22
Your age is: 22
```

Standard end line (endl)

The **endl** is a predefined object of **ostream** class. It is used to insert a new line characters and flushes the stream.

C++ Variable

A variable is a name of memory location. It is used to store data. Its value can be changed and it can be reused many times.

It is a way to represent memory location through symbol so that it can be easily identified. Let's see the syntax to

declare a variable:

1. `type variable_list;`

The example of declaring variable is given below:

1. `int x;`
2. `float y;`
3. `char z;`

Here, x, y, z are variables and int, float, char are data types.

We can also provide values while declaring the variables as given below:

1. `int x=5,b=10; //declaring 2 variable of integer type`
2. `float f=30.8;`
3. `char c='A';`



Rules for defining variables

- A variable can have alphabets, digits and underscore.
- A variable name can start with alphabet and underscore only. It can't start with digit.
- No white space is allowed within variable name.
- A variable name must not be any reserved word or keyword e.g. char, float etc.

Valid variable names:

1. `int a;`
2. `int _ab;`
3. `int a30;`

Invalid variable names:

1. `int 4;`
2. `int x y;`
3. `int double;`

C++ Data Types

A data type specifies the type of data that a variable can store such as integer, floating, character etc.



Data Types in C++

There are 4 types of data types in C++ language.



Types	Data Types
Basic Data Type	int, char, float, double, etc
Derived Data Type	array, pointer, etc
Enumeration Data Type	enum
User Defined Data Type	structure

Basic Data Types

The basic data types are integer-based and floating-point based. C++ language supports both signed and unsigned literals.

The memory size of basic data types may change according to 32 or 64 bit operating system.

Let's see the basic data types. Its size is given according to 32 bit OS.

Data Types	Memory Size	Range
Char	1 byte	-128 to 127
signed char	1 byte	-128 to 127
unsigned char	1 byte	0 to 127
Short	2 byte	-32,768 to 32,767
signed short	2 byte	-32,768 to 32,767
unsigned short	2 byte	0 to 32,767



HITEC UNIVERSITY, TAXILA

DEPARTMENT OF COMPUTER ENGINEERING

Lab # 01

Int	2 byte	-32,768 to 32,767
signed int	2 byte	-32,768 to 32,767
unsigned int	2 byte	0 to 32,767
short int	2 byte	-32,768 to 32,767
signed short int	2 byte	-32,768 to 32,767
unsigned short int	2 byte	0 to 32,767
long int	4 byte	
signed long int	4 byte	
unsigned long int	4 byte	
Float	4 byte	
Double	8 byte	
long double	10 byte	

C++ Keywords

A keyword is a reserved word. You cannot use it as a variable name, constant name etc. **A list of 32 Keywords in C++ Language which are also available in C language are given below.**

auto	break	case	char	const	continue	default	do
------	-------	------	------	-------	----------	---------	----



double	Else	enum	extern	float	for	goto	if
int	long	register	return	short	signed	sizeof	static
struct	switch	typedef	union	unsigned	void	volatile	while

A list of 30 Keywords in C++ Language which are not available in C language are given below.

asm	dynamic_cast	Namespace	reinterpret_cast	bool
explicit	new	static_cast	false	catch
operator	template	Friend	private	class
this	inline	Public	throw	const_cast
delete	mutable	Protected	true	try
typeid	typename	Using	virtual	wchar_t

C++ Operators

An operator is simply a symbol that is used to perform operations. There can be many types of operations like arithmetic, logical, bitwise etc.

There are following types of operators to perform different types of operations in C language.

- o Arithmetic Operators
- o Relational Operators
- o Logical Operators
- o Bitwise Operators



- o Assignment Operator
- o Unary operator
- o Ternary or Conditional Operator
- o Misc Operator

	Operator	Type
Binary Operator	+, -, *, /, %	Arithmetic Operators
	<, <=, >, >=, ==, !=	Relational Operators
	&&, , !	Logical Operators
	&, , <<, >>, ~, ^	Bitwise Operators
	=, +=, -=, *=, /=, %=	Assignment Operators
Unary Operator	→ ++, --	Unary Operator
Ternary Operator	→ ?:	Ternary or Conditional Operator

Precedence of Operators in C++

The precedence of operator specifies that which operator will be evaluated first and next. The associativity specifies the operators direction to be evaluated, it may be left to right or right to left.

Let's understand the precedence by the example given below: 1. **int**

```
data=5+10*10;
```

The "data" variable will contain 105 because * (multiplicative operator) is evaluated before + (additive operator).



HITEC UNIVERSITY, TAXILA

DEPARTMENT OF COMPUTER ENGINEERING

Lab # 01

The precedence and associativity of C++ operators is given below:

Category	Operator	Associativity
Postfix	() [] -> . ++ --	Left to right
Unary	+ - ! ~ ++ -- (type)* & sizeof	Right to left
Multiplicative	* / %	Left to right
Additive	+ -	Right to left
Shift	<< >>	Left to right
Relational	< <= > >=	Left to right
Equality	== !=	Right to left
Bitwise AND	&	Left to right
Bitwise XOR	^	Left to right
Bitwise OR		Right to left
Logical AND	&&	Left to right
Logical OR		Left to right
Conditional	?:	Right to left
Assignment	= += -= *= /= %= >>= <<= &= ^= =	Right to left
Comma	,	Left to right



C++ if-else

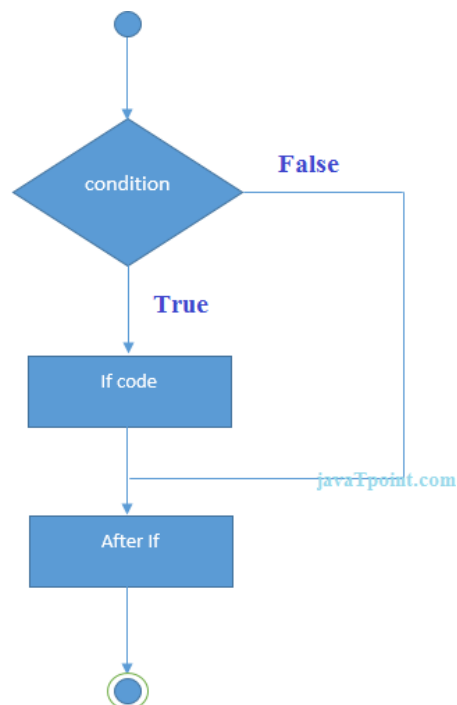
In C++ programming, if statement is used to test the condition. There are various types of if statements in C++.

- o if statement
- o if-else statement
- o nested if statement
- o if-else-if ladder

C++ IF Statement

The C++ if statement tests the condition. It is executed if condition is true.

1. **if**(condition){
2. //code to be executed
3. }





C++ If Example

```
1. #include <iostream>
2.     using namespace std;
3.
4. int main () {
5.     int num = 10;
6.     if (num % 2 == 0)
7.     {
8.         cout<<"It is even number";
9.     }
10.    return 0;
11. }
```

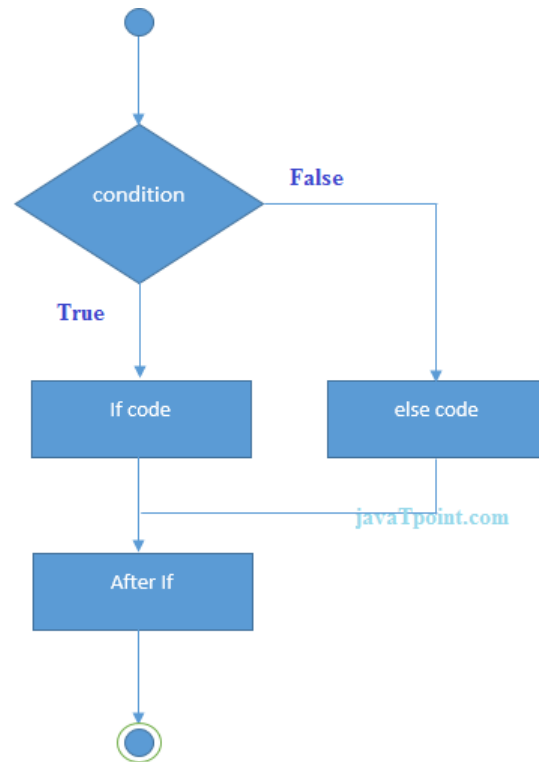
Output:/p>

It is even number

C++ IF-else Statement

The C++ if-else statement also tests the condition. It executes if block if condition is true otherwise else block is executed.

```
1. if(condition){
2.    //code if condition is true
3. }else{
4.    //code if condition is false
5. }
```



C++ If-else Example

```
1. #include <iostream>
2. using namespace std;
3. int main () {
4.     int num = 11;
5.     if (num % 2 == 0)
6.     {
7.         cout<<"It is even number";
8.     }
9.     else
10.    {
11.        cout<<"It is odd number";
12.    }
13.    return 0;
14. }
```



Output:

```
It is odd number
```

C++ If-else Example: with input from user

```
1. #include <iostream>
2. using namespace std;
3. int main () {
4.     int num;
5.     cout<<"Enter a Number: ";
6.     cin>>num;
7.     if (num % 2 == 0)
8.     {
9.         cout<<"It is even number"<<endl;
10.    }
11.    else
12.    {
13.        cout<<"It is odd number"<<endl;
14.    }
15.    return 0;
16. }
```

Output:

```
Enter a number:11
It is odd number
```

Output:

```
Enter a number:12
It is even number
```

C++ conditional statements

In C++ programming, if and switch statement is used to test the condition. There are various types of if statements in C++.

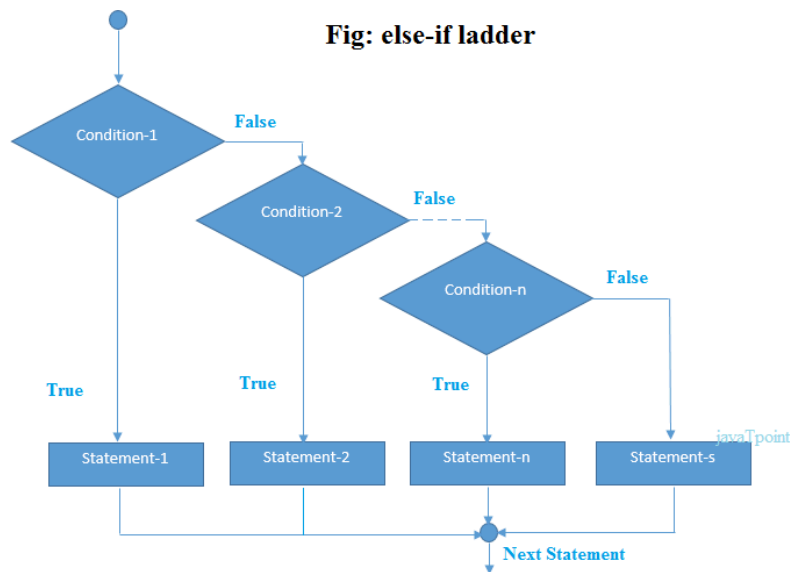


- o if statement
- o if-else statement
- o if-else-if ladder
- o nested if statement
- o switch statement

C++ IF-else-if ladder Statement

The C++ if-else-if ladder statement executes one condition from multiple statements.

1. **if**(condition1){
2. //code to be executed if condition1 is true
3. } **else if**(condition2){
4. //code to be executed if condition2 is true 5. }
6. **else if**(condition3){
7. //code to be executed if condition3 is true 8. }
9. ...
10. **else**{
11. //code to be executed if all the conditions are false
12. }





C++ If else-if Example

```
1. #include <iostream>
2. using namespace std;
3. int main () {
4.     int num;
5.     cout<<"Enter a number to check grade:";
6.     cin>>num;
7.     if (num <0 || num >100)
8.     {
9.         cout<<"wrong number";
10.    }
11.    else if(num >= 0 && num < 50){
12.        cout<<"Fail";
13.    }
14.    else if (num >= 50 && num < 60)
15.    {
16.        cout<<"D Grade";
17.    }
18.    else if (num >= 60 && num < 70)
19.    {
20.        cout<<"C Grade";
21.    }
22.    else if (num >= 70 && num < 80)
23.    {
24.        cout<<"B Grade";
25.    }
26.    else if (num >= 80 && num < 90)
27.    {
28.        cout<<"A Grade";
29.    }
30.    else if (num >= 90 && num <= 100)
31.    {
32.        cout<<"A+ Grade";
33.    }
34. }
```



Output:

```
Enter a number to check grade:66
C Grade
```

Output:

```
Enter a number to check grade:-2
wrong number
```

C++ nested IF-else Statement

In nested if-else, one if-else statement contains another if-else statement. Syntax

of nested if-else statement

```
1.  if(condition1){
2.      if(condition2){
3.          //code to be executed if condition1 and condition2 is true 4.  }
5.      else{
6.          //code to be executed if condition1 is true but condition2 is false 7.  }
8.  }
9.  else{
10.     if(condition3){
11.         //code to be executed if condition1 is false but condition3 is true
12.     }
13.     else{
14.         //code to be executed if condition1 and condition3 is false
15.     }
16. }
```



Example of nested if-else statement

```
1. #include<iostream>
2. using namespace std;

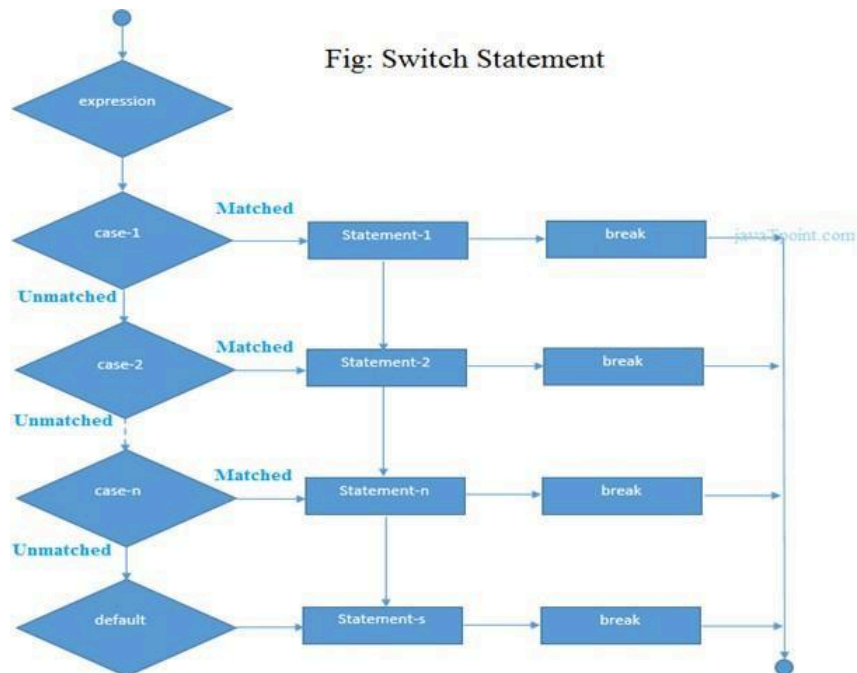
3. int
main() 4. {
5.     int age;
6.     cout<<"Enter your age: ";
7.     cin>>age;
8.     cout<<"Have you CNIC. Enter 1 for Yes and 0 for No: ";
9.     cin>>cnic;
10. //condition to check voting eligibility
11. if(age>=18)
12. {
13.     if(cnic==1)
14.         cout<<"You are eligible for voting"<<endl;
15.     else
16.     {
17.         cout<<"You are not eligible for voting because you don't have CNIC."<<endl;
18.     }
19. }
20. else
21. {
22.     if(age>=0)
23.         cout<<"You are not eligible for voting because you are underage."<<endl;
24.     else
25.     {
26.         cout<<"Invalid Inputs."<<endl;
27.     }
28. return 0;
29. }
```



C++ switch

The C++ switch statement executes one statement from multiple conditions. It is like if-else- if ladder statement in C++.

```
1. switch(expression){  
2.   case value1:  
3.       //code to be executed;  
4.       break  
5.  
6.   case value2:  
7.       //code to be executed;  
8.       break  
9.  
10. ....  
11.  
12. default:  
13.     //code to be executed if all cases are not matched;  
14.     break;  
15. }
```





C++ Switch Example

```
1. #include <iostream>
2. using namespace std;
3. int main () {
4.     int num;
5.     cout<<"Enter a number to check grade:";
6.     cin>>num;
7.     switch (num/10)
8.     {
9.         case 10:
10.            cout<<" YOUR GRADE IS A+";
11.            break;
12.         case 9:
13.            cout<<" YOUR GRADE IS A";
14.            break;
15.         case 8:
16.            cout<<" YOUR GRADE IS B";
17.            break;
18.         case 7:
19.            cout<<" YOUR GRADE IS C";
20.            break;
21.         case 6:
22.            cout<<" YOUR GRADE IS D";
23.            break;
24.         case 5:
25.            cout<<" YOUR GRADE IS F";
26.            break;
27.         case 4:
28.            cout<<" YOUR GRADE IS F";
29.            break;
30.         case 3:
31.            cout<<" YOUR GRADE IS F";
32.            break;
33.         case 2:
34.            cout<<" YOUR GRADE IS F";
```



```
35.         break;
36.         case 1:
37.             cout<<" YOUR GRADE IS F";
38.             break;
39.         case 0:
40.             cout<<" YOUR GRADE IS F";
41.             break;
42.         default:
43.             cout<<"Invalid Input";
44.             break;
45.     }
46. }
```

Output:

```
Enter a
number:
10
It is 10
```

Output:

```
Enter a number:
55
Not 10, 20 or 30
```

C++ Loops

There may be a situation, when you need to execute a block of code several number of times. In general, statements are executed sequentially: The first statement in a function is executed first, followed by the second, and so on.

Programming languages provide various control structures that allow for more complicated execution paths.

A loop statement allows us to execute a statement or group of statements multiple times and following is the general form of a loop statement in most of the programming languages

C++ programming language provides the following type of loops to handle looping requirements.

There are 3 type of loops in C++ Programming:



1. while Loop
2. do while Loop
3. for Loop

C++ while loop

A **while** loop statement repeatedly executes a target statement as long as a given condition is true.

Syntax

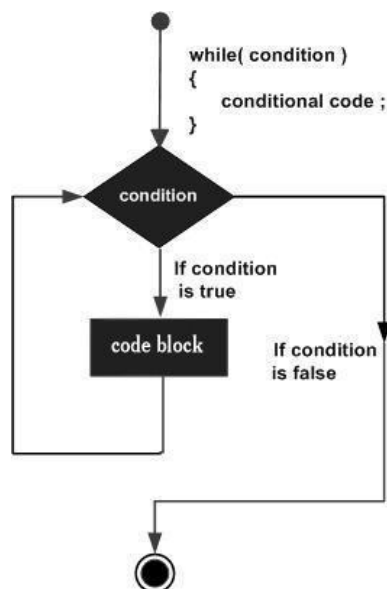
The syntax of a while loop in C++ is –

```
while(condition) {  
    statement(s);  
}
```

Here, **statement(s)** may be a single statement or a block of statements. The **condition** may be any expression, and true is any non-zero value. The loop iterates while the condition is true.

When the condition becomes false, program control passes to the line immediately following the loop.

Flow Diagram





Here, key point of the *while* loop is that the loop might not ever run. When the condition is tested and the result is false, the loop body will be skipped and the first statement after the while loop will be executed.

Example

```
#include <iostream>
using namespace std;

int main () {
    // Local variable declaration:
    int a = 10;

    // while loop execution
    while( a < 20 ) {
        cout << "value of a: " << a << endl;
        a++;
    }

    return 0;
}
```

When the above code is compiled and executed, it produces the following result:

```
value of a: 10 value
of a: 11 value of a:
12 value of a: 13
value of a: 14 value
of a: 15 value of a:
16 value of a: 17
value of a: 18 value
of a: 19
```

C++ do while loop

Unlike **for** and **while** loops, which test the loop condition at the top of the loop, the **do...while** loop checks its condition at the bottom of the loop.

A **do...while** loop is similar to a while loop, except that a do...while loop is guaranteed to execute at least one time.



Syntax

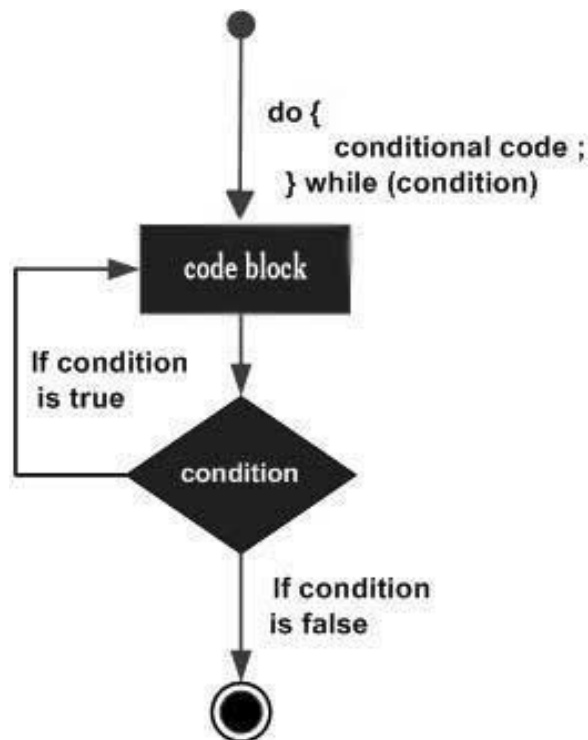
The syntax of a do...while loop in C++ is –

```
do { statement(s);  
}  
while( condition );
```

Notice that the conditional expression appears at the end of the loop, so the statement(s) in the loop execute once before the condition is tested.

If the condition is true, the flow of control jumps back up to do, and the statement(s) in the loop execute again. This process repeats until the given condition becomes false.

Flow Diagram





Example

```
#include <iostream>
using namespace std;

int main () {
    // Local variable declaration:
    int a = 10;

    // do loop execution
    do {
        cout << "value of a: " << a << endl;
        a = a + 1;
    } while( a < 20 );

    return 0;
}
```

When the above code is compiled and executed, it produces the following result:

value of a: 10 value
of a: 11 value of a:
12 value of a: 13
value of a: 14 value
of a: 15 value of a:
16 value of a: 17
value of a: 18 value
of a: 19

C++ for loop

A **for** loop is a repetition control structure that allows you to efficiently write a loop that needs to execute a specific number of times.

Syntax

The syntax of a for loop in C++ is –

for (init; condition; increment) { statement(s);

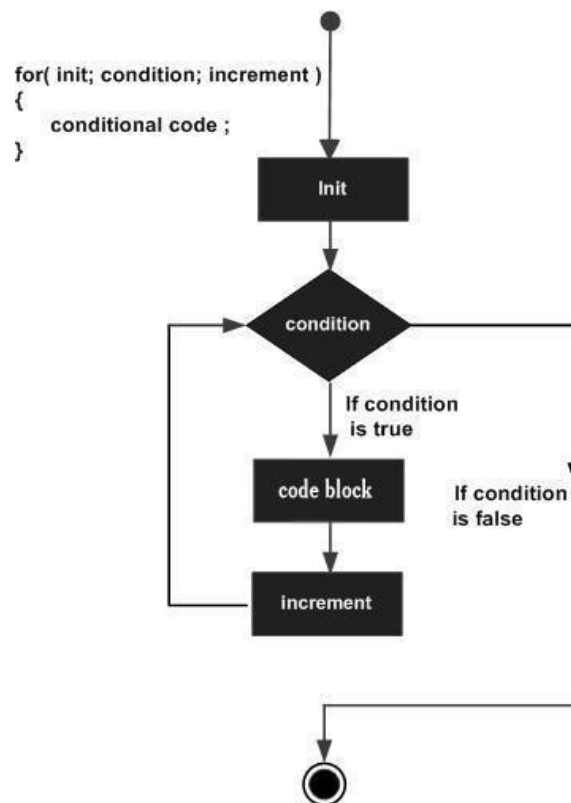


}

Here is the flow of control in a for loop:

- The **init** step is executed first, and only once. This step allows you to declare and initialize any loop control variables. You are not required to put a statement here, as long as a semicolon appears.
- Next, the **condition** is evaluated. If it is true, the body of the loop is executed. If it is false, the body of the loop does not execute and flow of control jumps to the next statement just after the for loop.
- After the body of the for loop executes, the flow of control jumps back up to the **increment** statement. This statement can be left blank, as long as a semicolon appears after the condition.
- The condition is now evaluated again. If it is true, the loop executes and the process repeats itself (body of loop, then increment step, and then again condition). After the condition becomes false, the for loop terminates.

Flow Diagram





Example

```
#include <iostream>
using namespace std;

int main () {
    // for loop execution
    for( int a = 10; a < 20; a = a + 1 ) {
        cout << "value of a: " << a << endl;
    }

    return 0;
}
```

When the above code is compiled and executed, it produces the following result:

value of a: 10 value
of a: 11 value of a:
12 value of a: 13
value of a: 14 value
of a: 15 value of a:
16 value of a: 17
value of a: 18 value
of a: 19

Lab Task 1:

Write and run a program that performs the following steps:

- **Take 10 temperature sensor values randomly between 0 and 50.**
- **Control room temperature by turning AC on and off according to threshold value=20.**

Lab Task 2:

Write a program to take 10 random times (only hours) and then print the message Good Morning, Good Afternoon, Good Evening Good Night and Good Midnight based on the following conditions

- **Midnight = 00:00**



- **Morning = 00:01 - 11:59**
- **Afternoon = 12:00 – 17:59**
- **Evening = 18:00 - 20:59**
- **Night = 21:00 - 23:59**

Also print 5 prayer reminder according to their specific times

Lab Performance Evaluation:

Read and practice the manual. Performance is evaluated at the end of lab based on successfully running and understanding of examples, tasks and viva using defined rubrics.

Lab Report Evaluation:

Submit the Lab Report by writing all the examples and tasks which must include the following:

1. Brief Description (3-5 lines)
2. The Code
3. The Results (Screenshot)

Important note: Lab Report must be according to the Lab Report Format available on Google Classroom and must be submitted on time. Two similar reports will get zero marks. So, don't try to copy your fellow reports. Lab report must be submitted in group of three maximum.